

Prácticas guiadas de TRIGGERS

PRÁCTICA 1: Trigger BEFORE INSERT (validación)

```
CREATE OR REPLACE TRIGGER trg_validar_salario
BEFORE INSERT ON empleados
FOR EACH ROW
BEGIN
    IF :NEW.salario < 1000 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Salario demasiado bajo');
    END IF;
END;
/
```

Prueba:

```
INSERT INTO empleados VALUES (4, 'Pedro', 900, 1, SYSDATE);
```

Puntos importantes:

- :NEW
- validación de política de negocio
- BEFORE → modifica o bloquea antes de insertar

PRÁCTICA 2: Trigger AFTER INSERT (auditoría)

Ver diferencia BEFORE vs AFTER

```
CREATE OR REPLACE TRIGGER trg_audit_insert
AFTER INSERT ON empleados
FOR EACH ROW
BEGIN
    INSERT INTO auditoria (accion, descripcion, fecha)
```

```
VALUES ('INSERT', 'Nuevo empleado: ' || :NEW.nombre, SYSDATE);  
  
END;  
  
/
```

Prueba:

```
INSERT INTO empleados VALUES (4, 'Pedro', 1500, 1, SYSDATE);
```

Puntos importantes:

- AFTER → ya ocurrió el cambio
- Ideal para auditoría

PRÁCTICA 3: Trigger BEFORE UPDATE con OLD vs NEW

Comparar valores:

```
CREATE OR REPLACE TRIGGER trg_control_salario  
  
BEFORE UPDATE OF salario ON empleados  
  
FOR EACH ROW  
  
BEGIN  
  
    IF :NEW.salario < :OLD.salario THEN  
  
        RAISE_APPLICATION_ERROR(-20002, 'No se puede bajar el salario');  
  
    END IF;  
  
END;  
  
/
```

Prueba:

```
UPDATE empleados SET salario = 1500 WHERE id_empleado = 1;
```

Puntos importantes:

- :OLD vs :NEW
- triggers en UPDATE

PRÁCTICA 4: Trigger AFTER DELETE

```
CREATE OR REPLACE TRIGGER trg_audit_delete  
  
AFTER DELETE ON empleados  
  
FOR EACH ROW  
  
BEGIN  
  
    INSERT INTO auditoria (accion, descripcion, fecha)  
  
    VALUES ('DELETE', 'Empleado eliminado: ' || :OLD.nombre, SYSDATE);  
  
END;  
  
/
```

PRÁCTICA 5: Trigger a nivel STATEMENT

```
CREATE OR REPLACE TRIGGER trg_statement_update  
  
AFTER UPDATE ON empleados  
  
BEGIN  
  
    INSERT INTO auditoria (accion, descripcion, fecha)  
  
    VALUES ('UPDATE MASIVO', 'Se ha hecho un UPDATE en empleados',  
SYSDATE);  
  
END;  
  
/
```

Prueba:

```
UPDATE empleados SET salario = salario + 100;
```

Punto importante:

- Solo se ejecuta **1 vez**, aunque afecte a muchas filas

PRÁCTICA 6: Trigger con múltiples eventos + IF INSERTING

```
CREATE OR REPLACE TRIGGER trg_multi_evento
BEFORE INSERT OR UPDATE OR DELETE ON empleados
FOR EACH ROW
BEGIN
    IF INSERTING THEN
        INSERT INTO auditoria VALUES (DEFAULT, 'INSERT', 'Insertado ' ||
:NEW.nombre, SYSDATE);

    ELSIF UPDATING THEN
        INSERT INTO auditoria VALUES (DEFAULT, 'UPDATE', 'Actualizado ' ||
:OLD.nombre, SYSDATE);

    ELSIF DELETING THEN
        INSERT INTO auditoria VALUES (DEFAULT, 'DELETE', 'Eliminado ' ||
:OLD.nombre, SYSDATE);

    END IF;
END;
```

/

Esto es CLAVE:

- INSERTING
- UPDATING
- DELETING

PRÁCTICA 7: Trigger con WHEN

```
CREATE OR REPLACE TRIGGER trg_salario_alto  
  
AFTER INSERT ON empleados  
  
FOR EACH ROW  
  
WHEN (NEW.salario > 3000)  
  
BEGIN  
  
    INSERT INTO auditoria (accion, descripcion, fecha)  
  
    VALUES ('ALERTA', 'Salario alto: ' || :NEW.nombre, SYSDATE);  
  
END;  
  
/
```

Puntos importantes:

- WHEN es SQL (no PL/SQL)
- No admite consultas

PRÁCTICA 8: Trigger INSTEAD OF (sobre vista)

Crea la siguiente vista:

```
CREATE VIEW vista_empleados AS  
  
SELECT e.id_empleado, e.nombre, d.nombre AS departamento  
  
FROM empleados e  
  
JOIN departamentos d ON e.id_departamento = d.id_departamento;
```

Crea el siguiente disparador:

```
CREATE OR REPLACE TRIGGER trg_instead_of_insert  
  
INSTEAD OF INSERT ON vista_empleados  
  
FOR EACH ROW  
  
BEGIN  
  
    INSERT INTO empleados (id_empleado, nombre, salario, id_departamento,  
    fecha_alta)  
  
    VALUES (:NEW.id_empleado, :NEW.nombre, 1500, 1, SYSDATE);  
  
END;  
  
/
```

Prueba:

```
INSERT INTO vista_empleados VALUES (10, 'Carlos', 'Informática');
```

Puntos importantes:

- Los triggers de sustitución (vista)
- Se ejecutan “en lugar de”

3. Actividad final integradora

Sistema de control empresarial. Debes crear:

1. Trigger que:
 - Evite salarios negativos
 - Control de subida máxima (ej: +20%)
2. Auditoría completa
 - Trigger multi-evento
3. Registro de borrados
4. Un trigger a nivel statement

4. Conceptos importantes

- No usar `COMMIT` dentro de triggers
- Problema de **tabla mutante**
- Orden de ejecución:
 1. BEFORE STATEMENT
 2. BEFORE ROW
 3. operación
 4. AFTER ROW
 5. AFTER STATEMENT
- Diferencia:
 - lógica de negocio → BEFORE
 - auditoría → AFTER
- BEFORE ROW → Validar – Bloquear
- AFTER ROW → Registrar lo que ha pasado
- STATEMENT → Registrar “evento global”

Errores típicos

- Usar `:OLD` en INSERT
- Usar `:NEW` en DELETE
- Pensar que STATEMENT se ejecuta por fila
- Intentar hacer `COMMIT`
- Duplicar lógica en muchos triggers